ASSIGNMENT 2 – CO2

NAME : RAJA RAGAVI R K

REGISTRATION : 1911260241

SUBJECT : PYTHON PROGRAMMING

COURSE CODE : CSA0822

GITHUB LINK: https://github.com/RajaRagavi16/CSA0822-1911260241-

Q1: Decisions with Many Boundaries

AIM:

Predict and compare subsidy decisions at all boundaries using separate if statements and an if-elif-else chain.

THEORY:

Separate if statements are independent, so more than one result can be produced; an if-elif-else chain selects one branch. The eight cases include 100, 101, 200 and 201 units. Nested conditions keep the senior exception clear; combined Boolean conditions are compact but can be harder to edit. A match-case version can use guards for ranges. A lookup through k ordered slabs takes O(k) time and O(1) extra space; a chain has the same worst-case bound.

PROGRAM CODE:

```
households=[(100,False,False),(101,True,False),(200,True,False),(201,False,False),(250,False,True),(99,False,False),(150,False,False),(220,True,False)]
def separate(units,card,senior):
 result=[]
 if units<=100: result.append('full')
 if 101<=units<=200 and card: result.append('partial')
 if units>200: result.append('none')
 if senior: result.append('senior subsidy')
 return result or ['no subsidy']
def chained(units,card,senior):
 if senior:return 'senior subsidy'
 if units<=100:return 'full'
 elif 101<=units<=200:return 'partial' if card else 'none'
 else:return 'none'
for h in households:print(h,'separate=',separate(*h),'if-elif=',chained(*h))
# For k slabs, chained tests take O(k) worst case and O(1) space.
# match-case or a sorted threshold loop also uses O(k) checks.
```

OUTPUT:

```
(100, False, False) separate= ['full'] if-elif= full
(101, True, False) separate= ['partial'] if-elif= partial
(200, True, False) separate= ['partial'] if-elif= partial
(201, False, False) separate= ['none'] if-elif= none
(250, False, True) separate= ['none', 'senior subsidy'] if-elif= senior subsidy
(99, False, False) separate= ['full'] if-elif= full
(150, False, False) separate= ['no subsidy'] if-elif= none
(220, True, False) separate= ['none'] if-elif= none
```

Q2: Loops that Count Attempts

AIM:

Verify OTP attempts with while and for loops, skip blank entries and lock after wrong attempts.

THEORY:

A while loop is natural when the attempt limit may change; for range(limit) is clear for a fixed limit. Blank input uses continue and does not increment the counter; break follows success. A loop else runs when no break occurs. pass does nothing and is unnecessary in the working logic. Updating the counter before checking blanks can wrongly consume attempts; failing to update it can cause an infinite loop. A correct security answer changes the limit from three to five. Recursion takes O(a) time and O(a) stack space; loops take O(a) time and O(1) space.

PROGRAM CODE:

```
def while_check(entries,correct,limit=3):
 attempts=0
 for value in entries:
  if not value.strip(): continue
  attempts+=1
  if value==correct: print('while: verified');break
  if attempts==limit: print('while: locked for 30 minutes');break
 else: print('while: attempts ended')
# A while variant updates attempts only after a nonblank entry.
def verify_while(entries,correct,limit=3):
 i=0;attempt=0
 while i<len(entries) and attempt<limit:
  value=entries[i];i+=1
  if not value.strip():continue
  attempt+=1
  if value==correct:print('while: verified');break
 else:print('while: locked for 30 minutes')
def verify_for(entries,correct,limit=3):
 values=(value for value in entries if value.strip())
 for _ in range(limit):
  value=next(values,None)
  if value is None:break
  if value==correct:print('for: verified');break
 else:print('for: locked for 30 minutes')
# If the security answer is correct, call with limit=5; pass is a no-op.
verify_while(['','111111','222222','333333'],'654321')
verify_for(['','111111','222222','333333'],'654321')
verify_for(['','111111','654321'],'654321',5)
print('recursive time O(a), stack O(a); loop stack O(1)')
```

OUTPUT:

```
while: locked for 30 minutes
for: locked for 30 minutes
for: verified
recursive time O(a), stack O(a); loop stack O(1)
```

Q3: Engineer a Vehicle Registration Number Validator

AIM:

Normalize typed registration numbers, validate their structure and format valid numbers for a receipt.

THEORY:

The format is two state letters, two district digits, one or two series letters and four digits. strip(), replace() and upper() create a normalized string; strings are immutable, so upper() does not alter its input. For 9 and 10 characters, s[-4:] selects the final digits and s[4:-4] selects the series. Slicing validation is O(L); a regular expression with fullmatch is also O(L) and expresses the pattern compactly.

PROGRAM CODE:

```
def normalise(text):return text.strip().replace(' ','').replace('-','').upper()
def is_valid(reg):
 if len(reg) not in (9,10):return False
 end=len(reg)-4
 return reg[:2].isalpha() and reg[2:4].isdigit() and reg[4:end].isalpha() and reg[end:].isdigit()
def format_reg(reg):
 end=len(reg)-4
 return f'{reg[:2]} {reg[2:4]} {reg[4:end]} {reg[end:]}'
samples=['tn 09 ab 1234','KA-01-M-0007','TN9AB1234','TN 22 1234 AB','MH12CD5678','XY 99 Z 0001']
for raw in samples:
 r=normalise(raw);print(raw,'->',r,format_reg(r) if is_valid(r) else 'INVALID')
s='TN09AB1234';print('immutable:',s.upper()!=s,'last 4:',s[-4:],'series:',s[4:-4])
# Regex alternative: re.fullmatch(r'[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}', r). O(L).
```

OUTPUT:

```
tn 09 ab 1234 -> TN09AB1234 TN 09 AB 1234
KA-01-M-0007 -> KA01M0007 KA 01 M 0007
TN9AB1234 -> TN9AB1234 INVALID
TN 22 1234 AB -> TN221234AB INVALID
MH12CD5678 -> MH12CD5678 MH 12 CD 5678
XY 99 Z 0001 -> XY99Z0001 XY 99 Z 0001
immutable: False last 4: 1234 series: AB
```

Q4: Building Long Text: + or join()?

AIM:

Build identical statement text with +=, join(), and an f-string per line, then compare average times.

THEORY:

Strings are immutable, so repeated += may copy accumulated prefixes; join combines parts in O(n) time in output length. CPython may optimize some += cases, so timings vary. Each completed string uses O(n) memory. Writing line by line or using io.StringIO streams output instead of keeping all parts and the final text at once. The code measures 1,000, 10,000 and 200,000 lines over three runs.

PROGRAM CODE:

```
import time
items=[f'Item{i}:Rs{i%50}' for i in range(200000)]
def plus(parts):
 text=''
 for p in parts:text+=p+';'
 return text
def join(parts):return ';'.join(parts)+(';' if parts else '')
def fstrings(parts):return ''.join(f'{p};' for p in parts)
for n in (1000,10000,200000):
 parts=items[:n];results=[]
 for fn in (plus,join,fstrings):
  ts=[]
  for _ in range(3):
   t=time.perf_counter();v=fn(parts);ts.append(time.perf_counter()-t)
  results.append((v,sum(ts)/3))
 print(n,'same:',results[0][0]==results[1][0]==results[2][0],'avg seconds:',*(round(x[1],4) for x in results))
print('join work O(output length); repeated += can copy prefixes, up to O(n^2).')
print('file writing/StringIO streams output and avoids retaining a second full string.')
```

OUTPUT:

```
1000 same: True avg seconds: 0.0001 0.0 0.0001
10000 same: True avg seconds: 0.0007 0.0001 0.0005
200000 same: True avg seconds: 0.3805 0.0034 0.0182
join work O(output length); repeated += can copy prefixes, up to O(n^2).
file writing/StringIO streams output and avoids retaining a second full string.
```

Q5: Functions, Parameters and Scope

AIM:

Explain the mutable-default bug and demonstrate independent orders, parameter styles, multiple return values and scope.

THEORY:

A default is evaluated once when the function is defined, so [] is shared across calls. None followed by a new list fixes it. Use positional arguments for simple order, keyword arguments for clarity, and defaults for optional values. Returning total and GST produces a tuple. Local names belong to a function; global names belong to the module and require global for rebinding; nonlocal refers to an enclosing function. Global mutation is usually avoided because it adds shared state. Passing a fresh list or using a closure also isolates each customer. Each append is O(1) amortized and an order of m items uses O(m) space.

PROGRAM CODE:

```
def add_item_buggy(item,order=[]):order.append(item);return order
def add_item(item,order=None):
 if order is None:order=[]
 order.append(item);return order
print('buggy:',add_item_buggy('Tea'),add_item_buggy('Vada'))
print('correct:',add_item('Tea'),add_item('Vada'))
for customer,item in enumerate(('Idli','Tea','Vada'),1):print(customer,add_item(item))
def bill(subtotal,gst_rate=.05):
 gst=subtotal*gst_rate
 return subtotal+gst,gst
print('positional/default:',bill(100),'keyword:',bill(subtotal=100,gst_rate=.10))
tax_rate=.05
def change_global():
 global tax_rate;tax_rate=.06
def scope_demo():
 total=10
 def add():
  nonlocal total;total+=2
 add();return total
change_global();print('global:',tax_rate,'nonlocal:',scope_demo(),'defaults:',add_item_buggy.__defaults__)
# Explicit list argument or a per-customer closure also avoids shared defaults.
```

OUTPUT:

```
buggy: ['Tea', 'Vada'] ['Tea', 'Vada']
correct: ['Tea'] ['Vada']
1 ['Idli']
2 ['Tea']
3 ['Vada']
positional/default: (105.0, 5.0) keyword: (110.0, 10.0)
global: 0.06 nonlocal: 12 defaults: (['Tea', 'Vada'],)
```

Q6: Recursion and Its Limits

AIM:

Count one-step and two-step stair-climbing routes, compare plain recursion with memoization, and choose a safe approach for 1,000 steps.

THEORY:

The recurrence is ways(n)=ways(n-1)+ways(n-2), with ways(0)=ways(1)=1. Plain recursion repeats work and has exponential time; memoization computes each n once in O(n) time and O(n) memory but still uses O(n) call stack. For n=5 the calls branch to n=4 and n=3, repeating smaller cases. A 1,000-deep recursive chain can exceed sys.getrecursionlimit(); raising the limit risks the process. An iterative loop is O(n) time and O(1) extra space, so it is the deployment choice.

PROGRAM CODE:

```
from functools import lru_cache
calls=0
def plain(n):
 global calls;calls+=1
 if n<=1:return 1
 return plain(n-1)+plain(n-2)
@lru_cache(None)
def memo(n):
 if n<=1:return 1
 return memo(n-1)+memo(n-2)
def iterative(n):
 a=b=1
 for _ in range(2,n+1):a,b=b,a+b
 return b if n else 1
print('ways 1..6:',[memo(n) for n in range(1,7)])
for n in (10,20,25,30):
 calls=0;plain(n);print(n,'plain calls:',calls,'memo calls:',memo.cache_info().currsize)
try:print('memo 1000:',memo(1000))
except RecursionError:print('memo 1000: RecursionError; limit:',__import__('sys').getrecursionlimit())
print('iterative 1000 digits:',len(str(iterative(1000))))
# Plain recursion exponential; memo O(n) time/space; iteration O(n) time O(1) extra.
```

OUTPUT:

```
ways 1..6: [1, 2, 3, 5, 8, 13]
10 plain calls: 177 memo calls: 7
20 plain calls: 21891 memo calls: 7
25 plain calls: 242785 memo calls: 7
30 plain calls: 2692537 memo calls: 7
memo 1000: 70330367711422815821835254877183549770181269836358732742604905087154537118196933579742249494562611733
  48775044924176599108818636326545022364710601205337412127386733911119813937312559876769009190224524532340350 1
iterative 1000 digits: 209
```